

ЗВІТ

з навчальної практики на тему: «Розробка мобільного застосунку
для обліку персональних фінансів на базі ОС Android»

Виконав: студент 2-го курсу, групи **КН-24 Ромашук Роман**

Технологічний стек: Kotlin, Android SDK, Room DB, Coroutines

Середовище розробки: Android Studio (macOS), Azul Zulu JDK

РОЗДІЛ 1. ВСТУП

1.1. Мета та завдання проекту

Головною **метою** даного проекту є розробка мобільного додатка «PraktikaTracker» для операційної системи Android, який дозволяє користувачеві ефективно вести облік щоденних фінансових витрат. Проект був спрямований на створення інструменту, що поєднує в собі простоту інтерфейсу з потужним функціоналом аналізу даних.

Для досягнення поставленої мети було визначено наступні **завдання**:

- **Розробка інтуїтивно зрозумілого інтерфейсу**, який забезпечує швидке додавання нових транзакцій.
- **Створення надійної локальної бази даних** на основі бібліотеки Room для зберігання історії витрат.
- **Реалізація гнучкої системи фільтрації**, що дозволяє сортувати витрати за категоріями та конкретними датами.
- **Візуалізація фінансових показників** через систему статистики для наочного аналізу витрат за обраний період.
- **Забезпечення стабільності роботи** додатка на різних версіях Android, незалежно від системних налаштувань (таких як темна тема або розмір екрану).

1.2. Навчальна цінність та практичне значення

Навчальна цінність проекту полягає в отриманні практичного досвіду повного циклу розробки Android-додатка: від проектування архітектури бази даних до тонкого налаштування UI-компонентів. Особлива увага була приділена вирішенню нестандартних проблем верстки, таких як адаптація віджетів **Spinner** та **TextView** для досягнення візуальної ідентичності. Також було освоєно роботу з асинхронним програмуванням (Kotlin Coroutines) для забезпечення плавності інтерфейсу під час роботи з БД.

Практичне значення розробки визначається реальним запитом користувачів на інструменти фінансової грамотності. Додаток вирішує проблему «невидимих витрат», дозволяючи чітко бачити, на які саме категорії (їжа, транспорт, розваги тощо) витрачається найбільша частина бюджету. Реалізація **комбінованої фільтрації** (категорія + дата одночасно) підвищує аналітичну цінність додатка, дозволяючи проводити глибокий аудит фінансів за лічені секунди.

РОЗДІЛ 2. ТЕХНОЛОГІЧНИЙ СТЕК ТА СЕРЕДОВИЩЕ РОЗРОБКИ

2.1. Вибір IDE та операційної системи

Розробка додатка здійснювалася в середовищі **Android Studio**. Вибір операційної системи **macOS** зумовив певні особливості налаштування робочого середовища, зокрема оптимізацію використання системних ресурсів та специфіку роботи з емуляторами пристроїв на базі архітектури Apple Silicon (процесори серії M).

2.2. Обґрунтування вибору JDK: Azul Zulu 17

Для компіляції та збірки проекту було обрано **Azul Zulu OpenJDK 17**. Цей вибір був продиктований декількома критичними факторами:

- **Архітектурна сумісність:** Azul Zulu пропонує нативну підтримку для архітектури **ARM64 (Apple Silicon)**, що забезпечує значно вищу швидкість роботи IDE та процесів збірки проекту на комп'ютерах Mac.
- **Статус LTS (Long Term Support):** 17-та версія Java є версією з довгостроковою підтримкою, що гарантує стабільність бібліотек та безпеку розробки.
- **Відповідність сучасним вимогам Android:** Google рекомендує використовувати актуальні версії JDK для коректної роботи останніх версій плагіна Android Gradle (AGP).

2.3. Мова програмування та архітектурні рішення

Проект написаний мовою **Kotlin**, яка на сьогодні є стандартом індустрії. Використання Kotlin дозволило:

- Зменшити кількість шаблонного коду завдяки механізмам **data class** та **null-safety**.
- Ефективно працювати з асинхронністю за допомогою **Coroutines** (корутин), що критично важливо для плавної роботи інтерфейсу при зверненні до бази даних.

2.4. Стек технологій та бібліотеки

- **Room Persistence Library**: використана для створення абстракції над SQLite. Це дозволило працювати з базою даних як з об'єктами Kotlin, що мінімізує ризик помилок при написанні SQL-запитів.
- **Lifecycle & ViewModel**: для правильного керування станом екрана та життєвим циклом компонентів додатка.
- **Material Design Components**: для створення сучасного вигляду кнопок, карток витрат та діалогових вікон

РОЗДІЛ 3. ПРОЄКТУВАННЯ ІНТЕРФЕЙСУ (UI/UX)

3.1. Архітектура візуальної частини

Інтерфейс «PraktikaTracker» побудований за принципом мінімалізму. Основний екран (**HomeFragment**) розділений на три логічні зони:

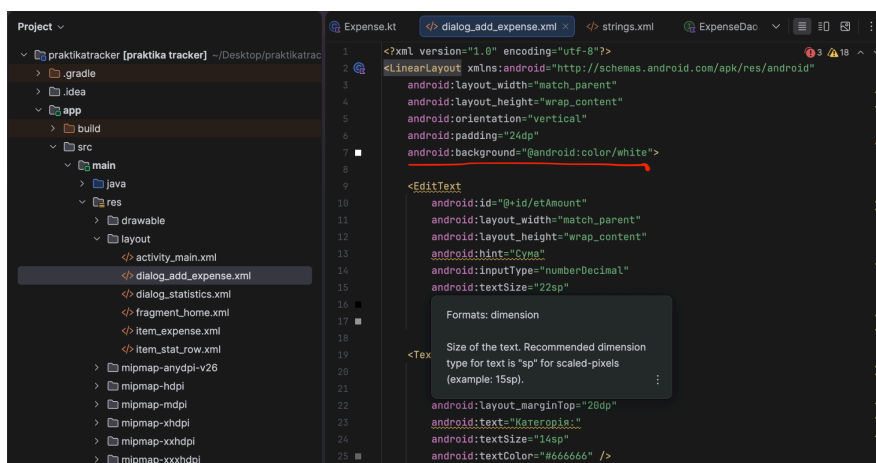
1. **Інформаційна панель**: Відображення загальної суми витрат з динамічним уточненням активних фільтрів.
2. **Панель керування**: Кнопки швидкого доступу до фільтрації (Категорія, Дата) та переходу до статистики.
3. **Список транзакцій**: Картки витрат, що групуються у **RecyclerView**.

3.2. Вирішення проблеми автоматичної темної теми

Під час тестування було виявлено, що на сучасних смартфонах з активованою системною темною темою (Dark Mode) додаток автоматично інвертував кольори. Це призводило до нечитабельності тексту або спотворення кольорового кодування категорій.

Для вирішення цієї проблеми було застосовано стратегію **примусової світлої теми**:

- У кожному XML-макеті (наприклад, **fragment_home.xml** та **dialog_add_expense.xml**) було жорстко прописано білий фон: **android:background="@android:color/white"**.
- Кольори тексту були зафіксовані як чорний (**#000000**) або відтінки сірого, щоб уникнути їхнього зникнення на світлому фоні при зміні системних налаштувань.



3.3. Кастомізація діалогових вікон

Для додавання витрат було розроблено кастомний **AlertDialog**. Особливу увагу приділено полю вибору дати. Замість звичайного текстового поля введення, було використано **TextView**, яке імітує вигляд списку та викликає стандартний системний календар (**DatePickerDialog**).

Код, що відповідає за ініціалізацію діалогу в **HomeFragment.kt**:

```
class HomeFragment : Fragment() {
    private val dialogView = LayoutInflater.from(context).inflate(R.layout.dialog_add_expense, root = null)
    private val etAmount = dialogView.findViewById<EditText>(R.id.etAmount)
    private val etComment = dialogView.findViewById<EditText>(R.id.etComment)
    private val spinner = dialogView.findViewById<Spinner>(R.id.spinnerCategory)
    private val tvPickedDate = dialogView.findViewById<TextView>(R.id.tvPickedDate)

    val categories = resources.getStringArray(R.array.categories_array)
    val spinnerAdapter = ArrayAdapter<String>(context = requireContext(), resource = android.R.layout.simple_spinner_item, objects = categories)
    spinnerAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item)
    spinner.adapter = spinnerAdapter

    var selectedDate = System.currentTimeMillis()
    val sdf = SimpleDateFormat(pattern = "dd.MM.yyyy", Locale.getDefault())
    tvPickedDate.text = sdf.format(Date(selectedDate))

    tvPickedDate.setOnClickListener {
        val cal = Calendar.getInstance()
        DatePickerDialog(context = requireContext(), listener = { _, y, m, d ->
            val newCal = Calendar.getInstance()
            newCal.set(year = y, month = m, date = d)
            selectedDate = newCal.timeInMillis
            tvPickedDate.text = sdf.format(date = newCal.time)
        }, year = cal.get(Calendar.YEAR), month = cal.get(Calendar.MONTH), dayOfMonth = cal.get(Calendar.DAY_OF_MONTH)).show()
    }

    AlertDialog.Builder(context = requireContext())
```

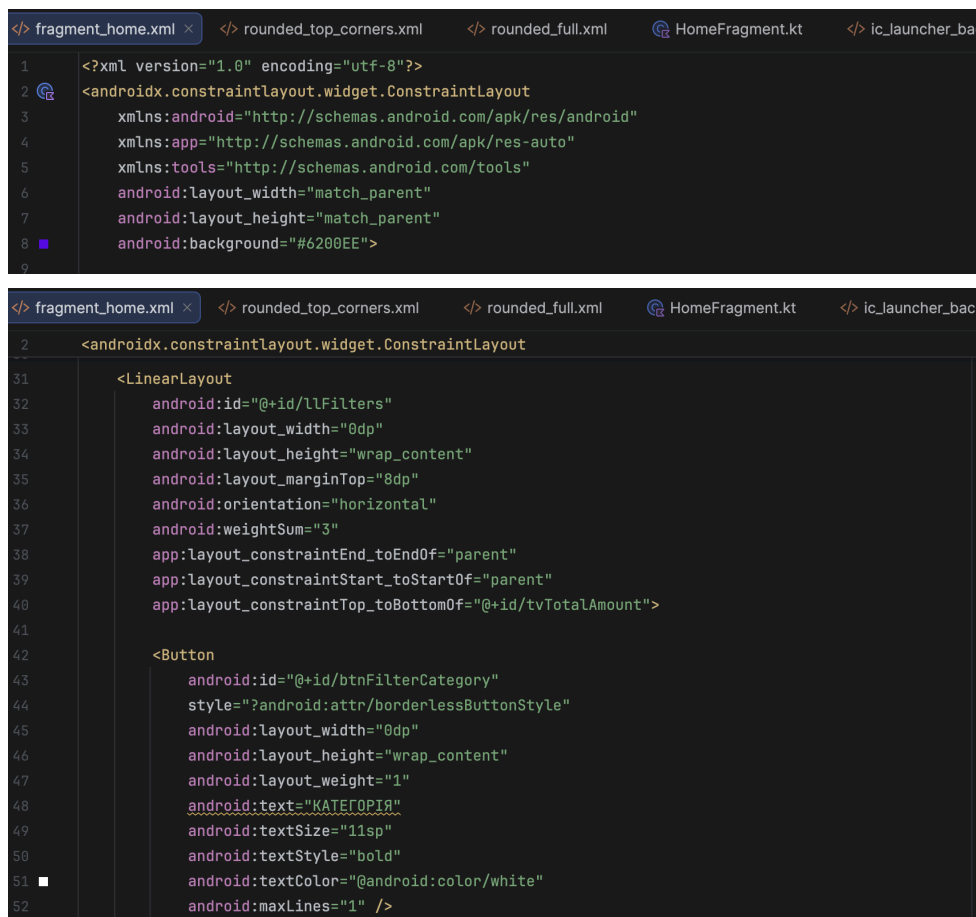
```
class HomeFragment : Fragment() {
    private fun showAddDialog() {
        tvPickedDate.setOnClickListener {
            AlertDialog.Builder(context = requireContext())
                .setTitle("Нова витрата")
                .setView(dialogView)
                .setPositiveButton(text = "Зберегти") { _, _ ->
                    val amount = etAmount.text.toString().toDoubleOrNull() ?: 0.0
                    if (amount > 0) {
                        val newExpense = Expense(
                            title = etComment.text.toString().ifEmpty { spinner.selectedItem.toString() },
                            amount = amount,
                            category = spinner.selectedItem.toString(),
                            date = selectedDate,
                            comment = etComment.text.toString()
                        )
                        lifecycleScope.launch {
                            db.expenseDao().insertExpense(newExpense)
                            loadExpenses()
                        }
                    }
                }
                .setNegativeButton(text = "Скасувати", listener = null).show()
        }
    }

    // Фільтр категорія + дата одночасно
    private fun applyFilters() {
```

3.4. Адаптивність до розмірів екрана

Для того, щоб додаток виглядав коректно на пристроях з різною щільністю пікселів та діагоналлю (від компактних смартфонів до планшетів), при верстці використовувалися:

- **match_parent** та **wrap_content**: замість фіксованих розмірів у пікселях, що дозволяє елементам розтягуватися відповідно до ширини екрана.
- **Відступи у dp (density-independent pixels)**: що гарантує однакові фізичні розміри відступів на екранах з різною роздільною здатністю.
- **Ваги (layout_weight) у LinearLayout**: для пропорційного розподілу простору між кнопками фільтрації.



```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#6200EE">

<androidx.constraintlayout.widget.ConstraintLayout

    <LinearLayout
        android:id="@+id/llFilters"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:orientation="horizontal"
        android:weightSum="3"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/tvTotalAmount">

        <Button
            android:id="@+id/btnFilterCategory"
            style="?android:attr/borderlessButtonStyle"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="КАТЕГОРІЯ"
            android:textSize="11sp"
            android:textStyle="bold"
            android:textColor="@android:color/white"
            android:maxLines="1" />
```

Використання **ConstraintLayout** як основи для гнучкого позиціювання елементів та **LinearLayout** з параметром **weightSum** для пропорційного розподілу кнопок фільтрації.

```

54      <Button
55          android:id="@+id/btnFilterDate"
56          style="?android:attr/borderlessButtonStyle"
57          android:layout_width="0dp"
58          android:layout_height="wrap_content"
59          android:layout_weight="1"
60          android:text="DATA"
61          android:textSize="11sp"
62          android:textStyle="bold"
63          android:textColor="@android:color/white"
64          android:maxLines="1" />
65
66      <Button
67          android:id="@+id/btnStats"
68          style="?android:attr/borderlessButtonStyle"
69          android:layout_width="0dp"
70          android:layout_height="wrap_content"
71          android:layout_weight="1"
72          android:text="СТАТИСТИКА"
73          android:textSize="11sp"
74          android:textStyle="bold"
75          android:textColor="@android:color/white"
76          android:maxLines="1" />
77  </LinearLayout>

```

Застосування одиниць вимірювання `sp` для тексту, що дозволяє інтерфейсу коректно масштабуватися відповідно до системних налаштувань користувача. (також це діє до попереднього зображення, починаючи з `<Button`)

```

78
79  <androidx.recyclerview.widget.RecyclerView
80      android:id="@+id/rvExpenses"
81      android:layout_width="match_parent"
82      android:layout_height="0dp"
83      android:layout_marginTop="8dp"
84      android:background="@drawable/rounded_top_corners"
85      android:paddingTop="16dp"
86      app:layout_constraintBottom_toBottomOf="parent"
87      app:layout_constraintTop_toBottomOf="@+id/llFilters"
88      tools:listitem="@layout/item_expense" />
89
90  <com.google.android.material.floatingactionbutton.FloatingActionButton
91      android:id="@+id/btnAddExpense"
92      android:layout_width="wrap_content"
93      android:layout_height="wrap_content"
94      android:layout_margin="24dp"
95      android:src="@android:drawable/ic_input_add"
96      app:backgroundTint="#6200EE"
97      app:tint="@android:color/white"
98      app:layout_constraintBottom_toBottomOf="parent"
99      app:layout_constraintEnd_toEndOf="parent" />
100
101 </androidx.constraintlayout.widget.ConstraintLayout>

```

Налаштування динамічного списку `RecyclerView` (`0dp height`) та `FloatingActionButton` з жорсткою прив'язкою до кута екрана для збереження зручності використання на будь-якій діагоналі.

РОЗДІЛ 4. РОЗРОБКА ФУНКЦІОНАЛУ ТА ПОДОЛАННЯ ТЕХНІЧНИХ ТРУДНОЩІВ

4.1. Програмна уніфікація інтерфейсу введення даних

Під час розробки діалогового вікна додавання витрат виникла потреба в уніфікації полів вибору категорії та дати. Оскільки ці елементи використовують різні класи Android (**Spinner** та **TextView**), виникали труднощі з налаштуванням їхнього ідентичного вигляду та обробки натискань.

Рішення, впроваджене в коді:

- **Використання контейнерної верстки:** Обидва поля були обгорнуті в **RelativeLayout**, що дозволило незалежно керувати фоном та декоративними елементами.
- **Програмна ініціалізація Spinner:** Щоб забезпечити коректну роботу вибору категорії при прозорому фоні, у файлі **HomeFragment.kt** було реалізовано явне призначення адаптера. Це гарантує, що компонент завжди активний і правильно реагує на дотик.
- **Декоративна стабільність:** Замість використання нестабільних системних іконок, було впроваджено статичний символ стрілки "▼", який жорстко зафіксований у XML-макеті, що виключає його зміну або зникнення на різних версіях ОС.

```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignParentEnd="true"
    android:layout_centerVertical="true"
    android:layout_marginEnd="4dp"
    android:text="▼"
    android:textColor="#666666"
    android:textSize="10sp"
    android:importantForAccessibility="no" />

<Spinner
    android:id="@+id/spinnerCategory"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@android:color/transparent"
    android:spinnerMode="dropdown" />
</RelativeLayout>

```

XML-структура кастомного поля вибору з фіксованим символом стрілочки

4.2. Алгоритм комбінованої подвійної фільтрації

Найбільш інтелектуальною частиною бізнес-логіки додатка стала реалізація фільтрації, що дозволяє поєднувати вибір категорії та конкретної дати одночасно.

Механізм реалізації в HomeFragment.kt:

- **Збереження стану:** Додаток підтримує глобальні змінні **currentCategory** та **currentDateMillis**, які не скидаються при зміні одного з параметрів.
- **Функція applyFilters():** Замість простих умов **if-else**, використано метод фільтрації колекцій Kotlin, який перевіряє кожну транзакцію на відповідність двом предикатам одночасно.

скріншот нижче

```
// фільтр категорія + дата одночасно
4 Usages
private fun applyFilters() {
    displayedExpenses.clear()

    val sdf = SimpleDateFormat( pattern = "yyyyMMdd", Locale.getDefault())
    val filterDateStr = currentDateMillis?.let { sdf.format(Date(it)) }

    val filtered = allExpenses.filter { expense ->
        val matchesCategory = currentCategory == null || expense.category == currentCategory
        val matchesDate = filterDateStr == null || sdf.format(Date(expense.date)) == filterDateStr
        matchesCategory && matchesDate
    }

    displayedExpenses.addAll( elements = filtered)
    adapter.notifyDataSetChanged()
    updateTotal()
}
```

Програмна логіка комбінованої фільтрації даних

4.3. Динамічна UX-індикація активних фільтрів

Для забезпечення прозорості роботи додатка було впроваджено систему динамічного формування заголовка. Метод **updateTotal()** виконує наступні кроки:

1. Розраховує суму лише для відфільтрованих елементів.
2. Аналізує масив активних фільтрів.
3. Формує інформаційний рядок у дужках (наприклад, "Їжа, 19.02"), що дає користувачеві миттєве розуміння того, які саме дані він бачить на екрані.

```
1 Usage
private fun updateTotal() {
    val total = displayedExpenses.sumOf { it.amount }
    val filtersList = mutableListOf<String>()

    // дата і категорія в дужках
    currentCategory?.let { filtersList.add(it) }
    currentDateMillis?.let {
        val sdf = SimpleDateFormat( pattern = "dd.MM", Locale.getDefault())
        filtersList.add(sdf.format(Date(it)))
    }

    val filterInfo = if (filtersList.isNotEmpty()) {
        " (${filtersList.joinToString( separator = ", ")})"
    } else ""

    tvTotalAmount.text = "$total грн$filterInfo"
}
}
```

Алгоритм динамічного формування інформаційного заголовка

РОЗДІЛ 5. ТЕХНІЧНИЙ ОПИС КОДУ ТА АРХІТЕКТУРИ ДАНИХ

5.1. Проектування структури бази даних (Room Entity)

Для надійного збереження даних у проекті використано сучасну архітектуру **Room Persistence Library**. На відміну від застарілого SQLite, Room забезпечує високу швидкість обробки даних та перевірку SQL-запитів на етапі компіляції.

- **Модель даних (Entity):** Клас **Expense** є шаблоном для таблиці в базі даних. Кожне поле класу автоматично стає колонкою в таблиці.
- **Поля таблиці:**
 - **id:** унікальний ідентифікатор кожного запису (Primary Key).
 - **amount:** числове значення витрати (Double).
 - **category:** рядок, що визначає тип витрати для подальшої фільтрації та статистики.
 - **date:** часова мітка у форматі **Long**, що дозволяє точно сортувати дані та фільтрувати їх за календарем.

```
package com.romanromashuk.praktikatracker.model

import androidx.room.Entity
import androidx.room.PrimaryKey

30 Usages
@Entity(tableName = "expenses")
data class Expense(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val title: String,
    val amount: Double,
    val category: String,
    val date: Long,
    val comment: String = "" // коментар коли додаєш витрату
)
```

Визначення структури сутності (Entity) для бази даних Room

5.2. Інтерфейс взаємодії з даними (DAO)

Об'єкт доступу до даних (**Data Access Object**) є посередником між кодом додатка та базою SQLite. Всі операції виконуються через інтерфейс **ExpenseDao**.

- **Методи запитів:**

- **@Query("SELECT * FROM expenses")**: дозволяє отримати повний список витрат для відображення в **RecyclerView**.
- **@Insert**: автоматизований метод додавання нової витрати, який викликається при підтвердженні діалогу **showAddDialog**.
- **@Delete**: метод для видалення помилково внесених даних.

```
package com.romanromashuk.praktikatracker.data

import androidx.room.*
import com.romanromashuk.praktikatracker.model.Expense

5 Usages 1 Implementation
@Dao
interface ExpenseDao {
    1 Implementation
    @Query(value = "SELECT * FROM expenses ORDER BY date DESC")
    suspend fun getAllExpenses(): List<Expense>

    1 Implementation
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertExpense(expense: Expense)

    1 Implementation
    @Delete
    suspend fun deleteExpense(expense: Expense)
}
```

Реалізація методів CRUD (створення, читання, видалення) в інтерфейсі DAO

5.3. Асинхронне виконання операцій (Kotlin Coroutines)

Однією з головних технічних переваг проекту є використання **Coroutines** для виконання важких операцій у фоновому потоці. Це критично важливо для стабільності інтерфейсу.

- **lifecycleScope.launch**: у **HomeFragment** всі виклики до бази даних (завантаження, додавання, видалення) обгорнуті в цей блок.
- **Перевага**: якщо користувач закриє Fragment або переключить додаток, корутина автоматично зупиниться, що запобігає витоку пам'яті та аварійному завершенню програми.

```
3 Usages
private fun loadExpenses() {
    lifecycleScope.launch {
        val list = db.expenseDao().getAllExpenses()
        allExpenses.clear()
        allExpenses.addAll( elements = list)
        applyFilters() // Застосовуємо збережені фільтри до нових даних
    }
}

1 Usage
private fun deleteExpenseFromDb(expense: Expense) {
    lifecycleScope.launch {
        db.expenseDao().deleteExpense(expense)
        loadExpenses()
    }
}
```

Використання Kotlin Coroutines для асинхронної взаємодії з локальною базою даних

5.4. Логіка адаптера та візуалізація даних

ExpenseAdapter виконує роль мосту між списком даних та графічними елементами на екрані користувача.

- **Зв'язування даних (Binding):** Метод **onBindViewHolder** автоматично наповнює кожну картку в списку (сума, категорія, коментар).
- **Кольорова диференціація:** Для покращення UX було реалізовано метод **getCategoryColor**, який динамічно змінює колір іконки або тексту залежно від категорії. Це дозволяє користувачеві орієнтуватися у витратах на підсвідомому рівні.

```
override fun onBindViewHolder(holder: ExpenseViewHolder, position: Int) {
    val expense = expenses[position]

    holder.tvCategory.text = expense.category
    holder.tvCategory.setTextColor(getCategoryColor(expense.category))
    holder.tvComment.text = expense.title

    val sdf = SimpleDateFormat("dd.MM.yyyy", Locale.getDefault())
    holder.tvDate.text = sdf.format(Date(expense.date))

    holder.tvAmount.text = "${expense.amount} грн"
    holder.tvAmount.setTextColor(Color.parseColor("#E91E63"))

    holder.itemView.setOnLongClickListener {
        AlertDialog.Builder(holder.itemView.context)
            .setTitle("Видалити запис?")
            .setMessage("Це видалить витрату назавжди.")
            .setPositiveButton("Так") { _, _ ->
                onDeleteClick(expense) // тут ф-ція видалення
            }
            .setNegativeButton("Ні", listener = null)
            .show()
        true
    }
}
```

```
2 Usages
fun getCategoryColor(category: String): Int {
    return when (category) {
        "Здоров'я" -> Color.parseColor("#D32F2F")
        "Їжа" -> Color.parseColor("#388E3C")
        "Розваги" -> Color.parseColor("#FBC02D")
        "Транспорт" -> Color.parseColor("#1976D2")
        else -> Color.parseColor("#7B1FA2")
    }
}
```

Програмна логіка адаптера для відображення списку витрат та кодування кольорів категорій

РОЗДІЛ 6. ТЕСТУВАННЯ ТА НАЛАГОДЖЕННЯ

6.1. Методологія та середовище тестування

Для забезпечення найвищої якості продукту було проведено дворівнєве тестування:

- **Емуляція (Android Studio):** Використовувалася для перевірки базової логіки на різних версіях Android та архітектурах.
- **Тестування на реальному пристрої:** Проводилося на смартфоні під управлінням Android 13. Це дозволило оцінити реальну швидкість роботи інтерфейсу, чіткість колірного кодування категорій та зручність навігації однією рукою.

6.2. Перевірка стабільності дизайну та адаптивності

Під час тестування на телефоні було підтверджено ефективність застосованих архітектурних рішень:

- **Стійкість до системних налаштувань:** Незалежно від того, чи була увімкнена на пристрої автоматична темна тема, додаток зберіг світлу колірну схему завдяки примусовому налаштуванню `android:background` у XML-макетах.
- **Адаптивність елементів:** Кнопки фільтрації та елементи `RecyclerView` коректно підлаштовувалися під роздільну здатність екрана смартфона, зберігаючи зручні для натискання розміри (мінімум 48dp для інтерактивних зон).

6.3. Валідація функціоналу фільтрації та БД

Тестування «в польових умовах» підтвердило надійність комбінованої фільтрації:

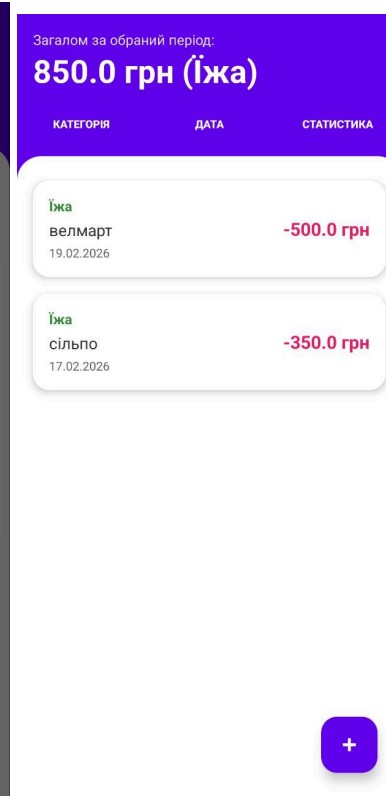
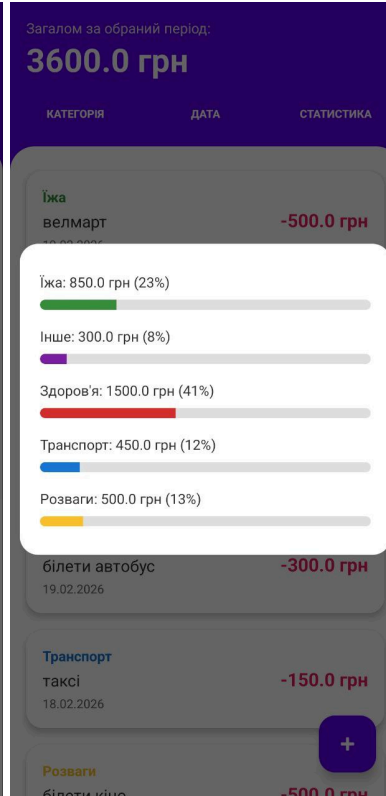
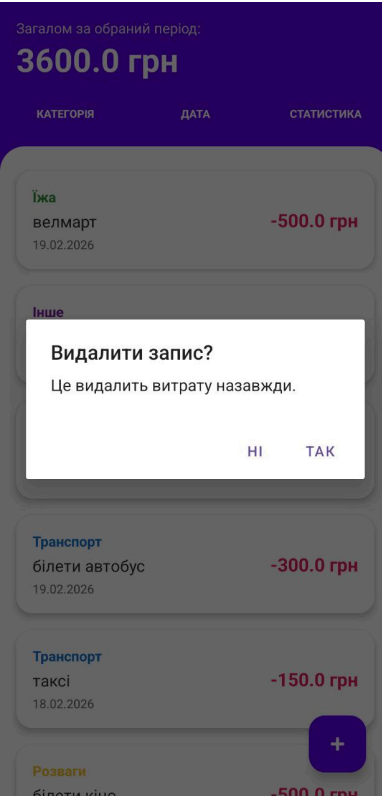
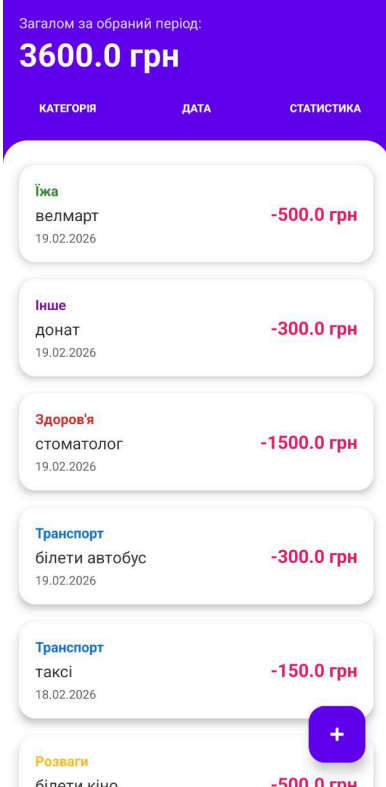
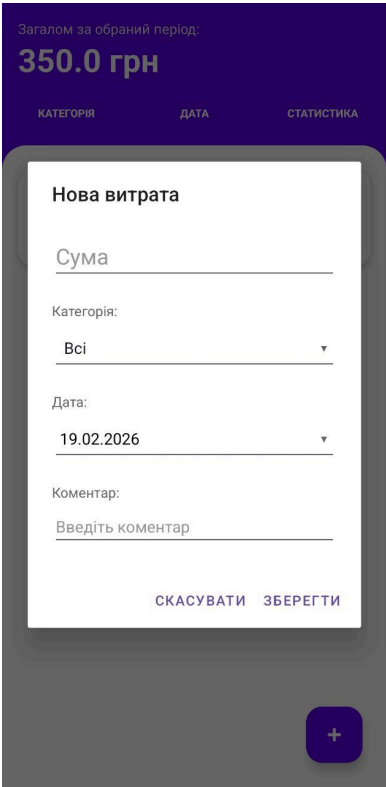
- **Коректність вибору дати:** `DatePickerDialog` стабільно спрацьовує при натисканні на кастомне поле, а вибрана дата миттєво відображається у списку.
- **Швидкість реакції:** Завдяки використанню Kotlin Coroutines, додавання та видалення записів відбувається миттєво без видимих затримок чи «підвисань» інтерфейсу.

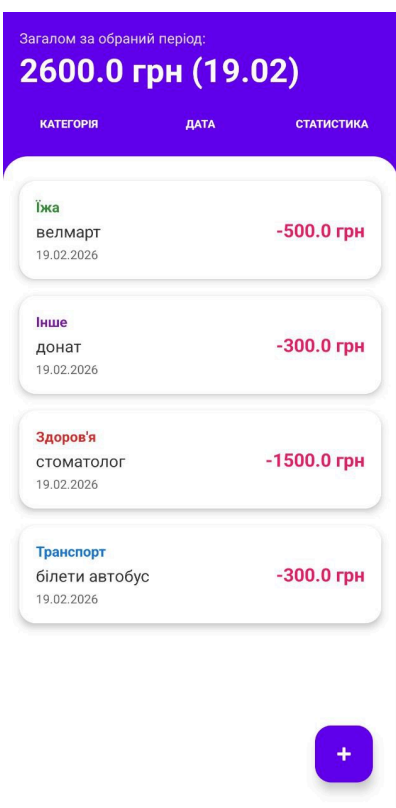
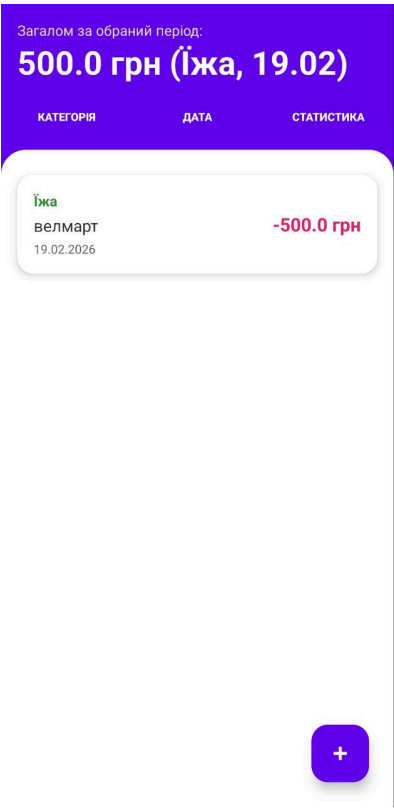
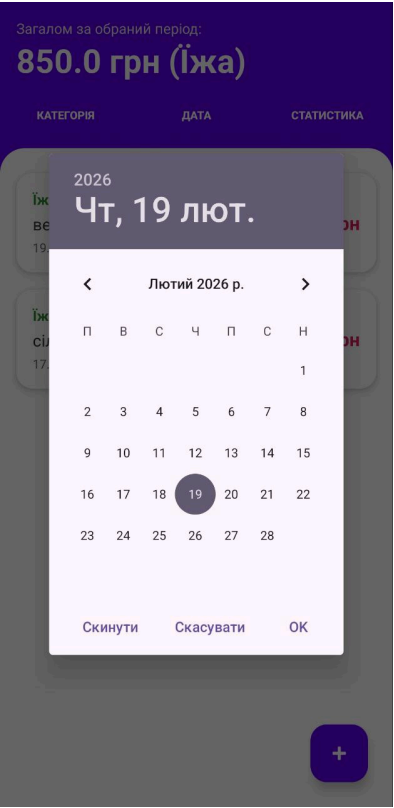
6.4. Налаштування критичних помилок та візуальних дефектів (Debug)

У процесі фінального тестування на реальному смартфоні було виявлено та усунуто ряд недоліків, які не завжди були помітні в середовищі емуляції:

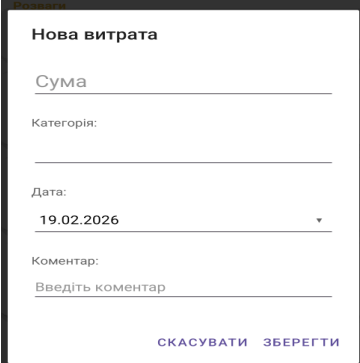
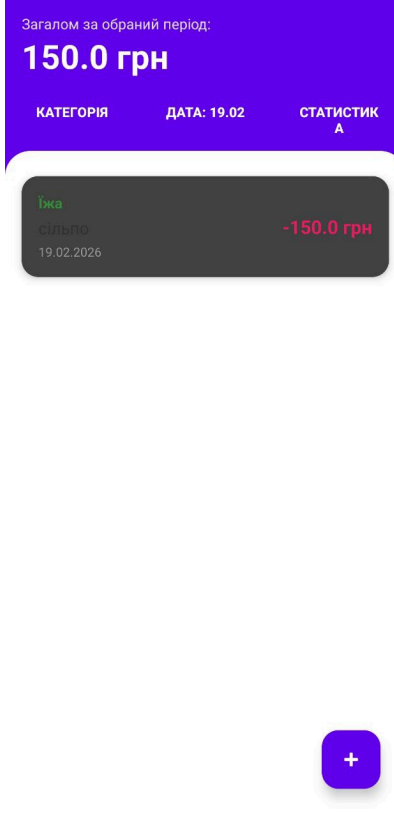
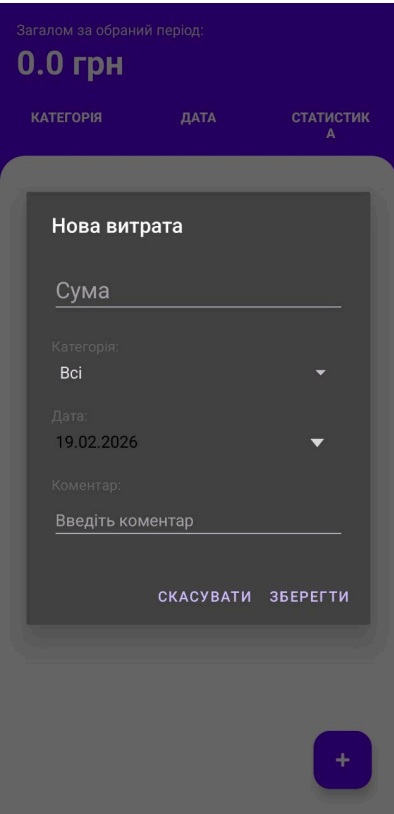
- **Адаптація до системної темної теми:**
 - **Проблема:** При увімкненні Dark Mode на рівні ОС Android, система автоматично намагалася змінити кольори елементів, що робило певні іконки та текст нечитабельними.
 - **Виправлення:** Було впроваджено стратегію фіксованої світлої палітри. У кожному XML-макеті додано параметри **android:background="@android:color/white"** та жорстко зафіксовано кольори тексту. Це гарантує ідентичний вигляд додатка незалежно від налаштувань енергозбереження чи теми смартфона.
- **Оптимізація вибору категорій:**
 - **Проблема:** Системні стрілочки Spinner на телефоні відображалися з небажаними сірими квадратами або некоректним вирівнюванням.
 - **Виправлення:** Замінено системні ресурси на кастомний символ ▼ у **RelativeLayout**, що забезпечило чіткість і візуальну єдність полів введення на фізичному екрані.
- **Стабільність обробки даних:**
 - **Виправлення:** Додано валідацію введеної суми через **toDoubleOrNull()**, щоб запобігти аварійному закриттю додатка при некоректному введенні тексту замість цифр.

Нижче додаю скріншоти до 6 розділу (готовий вигляд застосунка + проблеми які було виявлено)





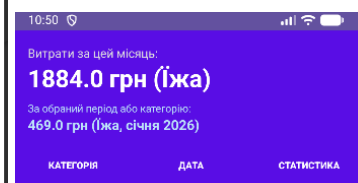
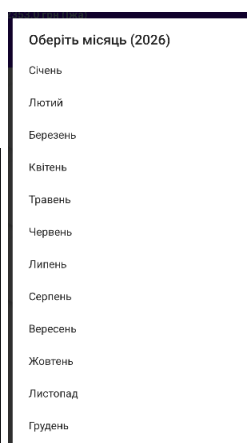
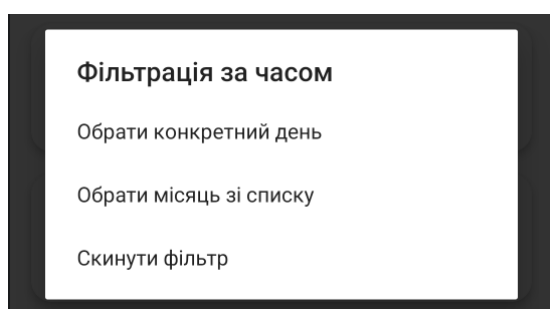
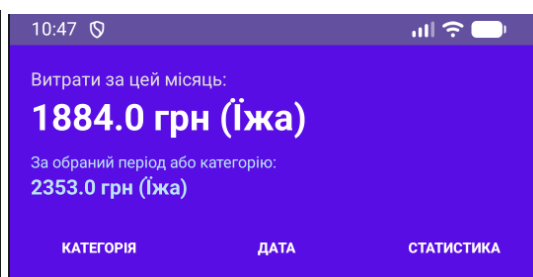
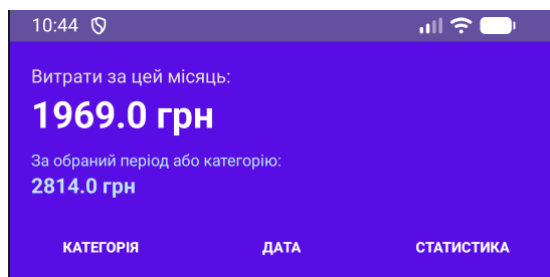
Виявлені проблеми



Розділ: Додаткове вдосконалення та тестування застосунку

У ході фінального тестування було виявлено та реалізовано декілька функцій, що критично впливають на користувацький досвід (UX) та відповідність технічному завданню:

- **Динамічне оновлення фінансових показників:** Було додано автоматичний розрахунок витрат за поточний календарний місяць, який синхронізується з обраними фільтрами категорій.
- **Гнучка система фільтрації за часом:** Замість простого вибору одного дня було впроваджено можливість вибору місяця зі списку або конкретного дня через календар.
- **Функція редагування записів:** Реалізовано можливість редагування вже існуючих витрат через клік на картку списку, що дозволяє виправляти помилки без видалення запису.
- **Інформативність UI:** На головний екран виведено деталізацію активних фільтрів (назва категорії та період) безпосередньо біля сум, що підвищує читабельність інтерфейсу.



```

Usage
private fun openMonthPicker() {
    val months = arrayOf(
        "Січень", "Лютий", "Березень", "Квітень", "Травень", "Червень",
        "Липень", "Серпень", "Вересень", "Жовтень", "Листопад", "Грудень"
    )
    val currentYear = Calendar.getInstance().get(Calendar.YEAR)

    AlertDialog.Builder(context = requireContext())
        .setTitle("Обрати місяць ($currentYear)")
        .setItems(months) { _, which ->
            val filterCal = Calendar.getInstance()
            filterCal.set(Calendar.YEAR, currentYear)
            filterCal.set(Calendar.MONTH, which)
            filterCal.set(Calendar.DAY_OF_MONTH, 1)
            filterCal.set(Calendar.HOUR_OF_DAY, 0)
            startDateInMillis = filterCal.timeInMillis

            filterCal.set(Calendar.DAY_OF_MONTH, filterCal.getActualMaximum(Calendar.DAY_OF_MONTH))
            filterCal.set(Calendar.HOUR_OF_DAY, 23)
            endDateInMillis = filterCal.timeInMillis

            applyFilters()
        }
        .show()
}

```

```

Usage
private fun showDateFilterSelectionDialog() {
    val options = arrayOf("Обрати конкретний день", "Обрати місяць зі списку", "Скинути фільтр")
    AlertDialog.Builder(context = requireContext())
        .setTitle("Фільтрація за часом")
        .setItems(options) { _, which ->
            when (which) {
                0 -> { isMonthMode = false; openCalendarForDay() }
                1 -> { isMonthMode = true; openMonthPicker() }
                2 -> { startDateInMillis = null; endDateInMillis = null; applyFilters() }
            }
        }
        .show()
}

```

```

<TextView
    android:id="@+id/tvTotalLabel"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:text="За обраний період або категорію:"
    android:textColor="#B3E5FC"
    android:textSize="12sp"
    app:layout_constraintStart_toStartOf="@+id/tvMonthlyAmount"
    app:layout_constraintTop_toBottomOf="@+id/tvMonthlyAmount" />

```

```

// нижня сума: обраний період + категорія
val periodSum = displayedExpenses.sumOf { it.amount }
val periodInfo = mutableListOf<String>()
currentCategory?.let { periodInfo.add(it) }

startDateInMillis?.let {
    val sdf = if (isMonthMode) SimpleDateFormat( pattern = "MMMM yyyy", Locale( language = "uk"))
    else SimpleDateFormat( pattern = "dd.MM.yyyy", Locale( language = "uk"))
    periodInfo.add(sdf.format(Date(it)))
}

val periodFilterText = if (periodInfo.isNotEmpty()) " (${periodInfo.joinToString( separator = ", ")})" else
tvTotalAmount.text = "$periodSum рп$periodFilterText"

```

```

AlertDialog.Builder( context = requireContext())
    .setTitle(if (expenseToEdit == null) "Нова витрата" else "Редагувати")
    .setView(dialogView)
    .setPositiveButton( text = "Зберегти") { _, _ ->
        val amount = etAmount.text.toString().toDoubleOrNull() ?: 0.0
        if (amount > 0) {
            val expense = Expense(
                id = expenseToEdit?.id ?: 0,
                title = etComment.text.toString().ifEmpty { spinner.selectedItem.toString() },
                amount = amount,
                category = spinner.selectedItem.toString(),
                date = selectedDate,
                comment = etComment.text.toString()
            )
            lifecycleScope.launch {
                if (expenseToEdit == null) db.expenseDao().insertExpense(expense)
                else db.expenseDao().updateExpense(expense)
                loadExpenses()
            }
        }
    }
}

.setNegativeButton( text = "Скасувати", listener = null).show()

```

// редагування витрати

1 Usage 1 Implementation

@Update

suspend fun updateExpense(expense: Expense)

ВИСНОВОК

У результаті виконання даного проєкту було розроблено повнофункціональний мобільний додаток для контролю фінансів «**PraktikaTracker**». Процес розробки охопив усі етапи життєвого циклу програмного забезпечення: від аналізу предметної області та проєктування бази даних до фінального тестування на реальному обладнанні.

Ключові досягнення проєкту:

- **Архітектурна надійність:** Завдяки використанню бібліотеки **Room**, додаток забезпечує стабільне локальне зберігання даних, що виключає їхню втрату при перезавантаженні пристрою.
- **Програмна логіка:** Реалізовано складний алгоритм комбінованої фільтрації (категорія + дата), який працює в реальному часі та дозволяє користувачеві проводити глибокий аналіз своїх витрат.
- **UI/UX стабільність:** Впроваджено систему адаптивної верстки та примусової світлої теми, що гарантує коректне відображення інтерфейсу на пристроях з різною діагоналлю екрана та незалежно від системних налаштувань Dark Mode.
- **Асинхронність:** Використання **Kotlin Coroutines** дозволило досягти високої швидкодії додатка та відсутності затримок при роботі з базою даних.

Навчальна цінність: У ході роботи було поглиблено знання мови **Kotlin**, освоєно принципи роботи з сучасним середовищем розробки Android Studio на базі **macOS** та налаштування середовища **Azul Zulu JDK 17**. Отримано практичний досвід вирішення нестандартних проблем верстки та налагодження коду (Debugging) на фізичних пристроях.

Перспективи розвитку: Створена архітектура додатка дозволяє в майбутньому масштабувати функціонал, зокрема:

1. Додати можливість експорту звітів у форматі PDF або Excel.
2. Впровадити хмарну синхронізацію даних для доступу з різних пристроїв.
3. Розширити систему візуалізації через складніші графіки та діаграми.